Reporte técnico: benchmark ConvNet image-only (3x2)

1. Objetivo

Documentar formalmente el benchmark `ConvNet image-only` ya ejecutado en el proyecto, cubriendo la matriz completa **3 modelos x 2 datasets** (6 corridas), con evidencia trazable en logs, resúmenes y manifiestos de ejecución.

2. Entorno de ejecución

- Configuración cargada desde `configs/convnets_image_only.yaml`.
- Dispositivo solicitado y resuelto: `mps` (Metal Performance Shaders).
- Modo de ejecución: `Eval-only mode: False` (entrenamiento + evaluación).
- Pipeline reportado por manifiestos: `ConvNetPipeline`.
- Entorno Python observado en rutas de warning: `.venv/lib/python3.11/...`.
- Ventana temporal de ejecución (UTC en manifiestos):
  - Inicio global: `2026-04-24T04:02:17.416685+00:00`
  - Fin global: `2026-04-24T04:51:54.584368+00:00`
  - Duración acumulada 6 corridas: **2977.10 s** (49m 37.10s)

Fuentes: `benchmark_convnets.log`, `outputs/**/run_manifest.json`.

3. Metodología

Se ejecutó un benchmark exhaustivo sobre la grilla:

- Datasets: `damage_dataset`, `crisisMMD`
- Modelos: `resnet50`, `efficientnet_b0`, `vgg16`

Estrategia aplicada:

1. Carga de dataset y splits predefinidos.
2. Entrenamiento por 6 epochs por combinación dataset-modelo.
3. Evaluación sobre conjunto de test al final de cada corrida.
4. Registro de métricas (`test_accuracy`, `test_loss`), tiempos y metadatos en `run_manifest.json`.
5. Agregación por dataset en `benchmark_summary.json`.

Nota metodológica:

- El benchmark base 3x2 mantiene **una corrida por combinación** (dataset-modelo).
- Adicionalmente, se ejecutó una validación multi-seed para `resnet50` y `efficientnet_b0` con seeds `42/43/44` (6 corridas adicionales por dataset, 12 en total).

4. Configuración usada

Configuración tomada de `configs/convnets_image_only.yaml`:

- Datasets:

    - `damage_dataset` (6 clases): `non_damage`, `fires`, `flood`, `damaged_infrastructure`, `damaged_nature`, `human_damage`
    - `crisisMMD` (2 clases): `informative`, `not_informative`
- Modelos y parámetros globales:
    - `image_size`: 224
    - `trainable_layers`: 2
    - `use_pretrained`: `true`
    - `epochs`: 6
    - `num_workers`: 2
    - `lr`: 2.0e-4
    - `weight_decay`: 1.0e-4
    - `batch_size`: 16 (`resnet50`, `vgg16`) / 32 (`efficientnet_b0`)
- Dispositivo configurado: `mps`
- Salida base: `outputs/`

Tamaños de split (desde `run_manifest.json`):

Dataset	Train	Val	Test	Clases
damage_dataset	4761	530	588	6
crisisMMD	13608	2237	2237	2

5. Resultados por dataset y modelo

5.1 `damage_dataset`

Modelo	Run ID	Test Accuracy	Test Loss	Duración (s)
resnet50	`20260424T040217416685Z`	**0.8656**	0.4388	376.52
efficientnet_b0	`20260424T040833934478Z`	0.8554	**0.4068**	**278.55**
vgg16	`20260424T041312489094Z`	0.8197	0.4989	437.18

Mejor modelo por accuracy en `damage_dataset`: **resnet50**.

5.2 `crisisMMD`

Modelo	Run ID	Test Accuracy	Test Loss	Duración (s)
resnet50	`20260424T042029731313Z`	**0.7380**	**0.5145**	613.48
efficientnet_b0	`20260424T043043211276Z`	0.7318	0.5318	**335.91**
vgg16	`20260424T043619126596Z`	0.7045	0.5731	935.46

Mejor modelo por accuracy en `crisisMMD`: **resnet50**.

Fuentes: `outputs/damage_dataset/benchmark_summary.json`, `outputs/crisisMMD/benchmark_summary.json`, `outputs/**/run_manifest.json`.

6. Análisis comparativo

- **Rendimiento por accuracy**: `resnet50` lidera en ambos datasets.
- **Gap de accuracy vs segundo lugar**:
  - `damage_dataset`: `resnet50` supera a `efficientnet_b0` por **+0.0102** (+1.02 pp).
  - `crisisMMD`: `resnet50` supera a `efficientnet_b0` por **+0.0063** (+0.63 pp).
- **Costo temporal**:
  - `efficientnet_b0` es el más rápido en ambos datasets (278.55 s y 335.91 s).
  - `vgg16` es el más lento y además el de menor accuracy en ambos datasets.
- **Patrón de dificultad por dataset**:
  - Todos los modelos bajan accuracy al pasar de `damage_dataset` a `crisisMMD`.
  - Caídas de accuracy (`damage_dataset` -> `crisisMMD`):
    - `resnet50`: -0.1276
    - `efficientnet_b0`: -0.1237
    - `vgg16`: -0.1152

7. Hallazgos principales

1. El benchmark **3x2 está completo** y trazable por `run_id` en manifiestos y resúmenes.
2. **`resnet50` es el mejor baseline por accuracy** en ambos datasets.
3. `efficientnet_b0` muestra el mejor compromiso tiempo/rendimiento (segundo en accuracy y más rápido).
4. `vgg16` queda dominado en este setup: menor accuracy y mayor tiempo de ejecución.
5. `crisisMMD` resulta más desafiante para el enfoque image-only en las tres arquitecturas.

8. Limitaciones / advertencias

- La validación multi-seed se ejecutó para `resnet50` y `efficientnet_b0`, pero **no** para `vgg16`; por tanto, la comparación de estabilidad entre seeds no cubre las tres arquitecturas.
- El benchmark reporta métricas de test globales (`test_accuracy`, `test_loss`), sin intervalos de confianza.
- En logs de `crisisMMD` aparece warning recurrente de PIL sobre imágenes palette con transparencia (`convert to RGBA`), potencialmente relevante para consistencia de preprocesado.
- Este reporte no incorpora métricas adicionales (p. ej. F1 macro como criterio principal), aunque los logs incluyen classification reports.

9. Consolidación multi-seed (post benchmark base)

Se consolidó la validación multi-seed (`42/43/44`) de `resnet50` y `efficientnet_b0` sobre la configuración base (`configs/convnets_image_only.yaml`), usando como fuente primaria los logs de seed dedicados.

9.1 Accuracy por seed

Dataset	Seed	resnet50 (Acc)	efficientnet_b0 (Acc)	Gap (resnet50 - efficientnet_b0)
damage_dataset	42	**0.8810**	0.8486	+0.0324

Dataset	Seed	resnet50 (Acc)	efficientnet_b0 (Acc)	Gap (resnet50 - efficientnet_b0)
damage_dataset	43	**0.8724**	0.8316	+0.0408
damage_dataset	44	**0.8673**	0.8401	+0.0272
crisisMMD	42	**0.7407**	0.7389	+0.0018
crisisMMD	43	**0.7452**	0.7322	+0.0130
crisisMMD	44	**0.7465**	0.7398	+0.0067

9.2 Resumen de estabilidad

Dataset	Modelo	Mean Acc	Rango Acc (max-min)	Std Acc (poblacional)	Mean Macro-F1
damage_dataset	**resnet50**	**0.8736**	0.0136	0.0056	**0.8233**
damage_dataset	efficientnet_b0	0.8401	0.0170	0.0069	0.7900
crisisMMD	**resnet50**	**0.7442**	0.0058	0.0025	**0.7300**
crisisMMD	efficientnet_b0	0.7370	0.0076	0.0034	0.7200

Interpretación:

- `resnet50` gana en accuracy en los 6 pares comparables (2 datasets x 3 seeds).
- La variación por seed es baja en ambos modelos; `resnet50` mantiene ventaja en promedio y con dispersión similar o menor.
- La diferencia es más marcada en `damage_dataset`; en `crisisMMD` el gap es menor pero consistente a favor de `resnet50`.

10. Micro-tuning de `efficientnet_b0`

Se ejecutó una fase corta de micro-tuning de `efficientnet_b0` con variantes A/B/C, siempre sobre el mismo pipeline y datasets.

10.1 Definición de variantes

- **Variante B** (`configs/convnets_image_only_efficientnet_b0_tuning_B.yaml`): `epochs=4`, resto igual al baseline.
- **Variante A** (`configs/convnets_image_only_efficientnet_b0_tuning_A.yaml`): `trainable_layers=4`, `epochs=6`, `lr=2e-4`, `weight_decay=1e-4`.
- **Variante C** (`configs/convnets_image_only_efficientnet_b0_tuning_C.yaml`): `trainable_layers=2`, `epochs=6`, `lr=1e-4`, `weight_decay=3e-4`.

10.2 Resultados observados

Variante	Seed	damage_dataset Acc / Loss	crisisMMD Acc / Loss	Lectura breve
B	42	0.8486 / 0.4293	0.7389 / 0.5241	Equivalente al baseline seed 42 (sin mejora).
A	42	**0.8673** / **0.4092**	**0.7430** / 0.5316	Mejora de accuracy en ambos datasets para este seed.
A	43	0.8316 / 0.4416	0.7389 / 0.5294	No replica la mejora en `damage_dataset`; mejora acotada en `crisisMMD` vs seed 43 base.
C	42	0.8452 / 0.4265	**0.7430** / **0.5183**	Trade-off: leve caída en `damage_dataset`, ganancia en `crisisMMD`.

10.3 Cierre de la línea de micro-tuning

- No aparece una mejora robusta y consistente entre seeds/datasets que justifique escalar esta línea en esta etapa.
- La ganancia más alta (variante A seed 42) no se sostuvo al repetir con otro seed (A seed 43).
- Se cierra el micro-tuning de `efficientnet_b0` por ahora y se prioriza estabilidad del baseline principal.

11. Recomendación final actualizada

Con la evidencia conjunta (benchmark base + multi-seed + micro-tuning):

- **Baseline final**: `resnet50`.
- **Alternativa operativa**: `efficientnet_b0` cuando el costo temporal sea prioridad.
- **Estado de micro-tuning `efficientnet_b0`**: cerrado por ahora (sin escalamiento inmediato).

12. Próximos pasos actualizados

1. Mantener `resnet50` como referencia para comparativas futuras (nuevos backbones o cambios de pipeline).
2. Si se reabre tuning de `efficientnet_b0`, hacerlo con diseño mínimo multi-seed por variante (>=3 seeds) para evitar decisiones por seed único.
3. Extender multi-seed a `vgg16` o reemplazarlo por una arquitectura más competitiva para cerrar comparación de estabilidad de alternativas.
4. Consolidar reporte complementario con métricas por clase y macro-F1 como criterio secundario de selección.

Evidencia y trazabilidad (archivos fuente)

- `benchmark_convnets.log`
- `configs/convnets_image_only.yaml`
- `outputs/damage_dataset/resnet50/run_manifest.json`

- `outputs/damage_dataset/efficientnet_b0/run_manifest.json`
- `outputs/damage_dataset/vgg16/run_manifest.json`
- `outputs/crisisMMD/resnet50/run_manifest.json`
- `outputs/crisisMMD/efficientnet_b0/run_manifest.json`
- `outputs/crisisMMD/vgg16/run_manifest.json`
- `seed_resnet50_42.log`
- `seed_resnet50_43.log`
- `seed_resnet50_44.log`
- `seed_efficientnet_b0_42.log`
- `seed_efficientnet_b0_43.log`
- `seed_efficientnet_b0_44.log`
- `tuning_efficientnet_B.log`
- `tuning_efficientnet_A.log`
- `tuning_efficientnet_A_seed43.log`
- `tuning_efficientnet_C_seed42.log`
- `configs/convnets_image_only_efficientnet_b0_tuning_A.yaml`
- `configs/convnets_image_only_efficientnet_b0_tuning_B.yaml`
- `configs/convnets_image_only_efficientnet_b0_tuning_C.yaml`
- `outputs/damage_dataset/resnet50/seed_42/run_manifest.json`
- `outputs/damage_dataset/resnet50/seed_43/run_manifest.json`
- `outputs/damage_dataset/resnet50/seed_44/run_manifest.json`
- `outputs/crisisMMD/resnet50/seed_42/run_manifest.json`
- `outputs/crisisMMD/resnet50/seed_43/run_manifest.json`
- `outputs/crisisMMD/resnet50/seed_44/run_manifest.json`
- `outputs/damage_dataset/efficientnet_b0/seed_42/run_manifest.json`
- `outputs/damage_dataset/efficientnet_b0/seed_43/run_manifest.json`
- `outputs/damage_dataset/efficientnet_b0/seed_44/run_manifest.json`
- `outputs/crisisMMD/efficientnet_b0/seed_42/run_manifest.json`
- `outputs/crisisMMD/efficientnet_b0/seed_43/run_manifest.json`
- `outputs/crisisMMD/efficientnet_b0/seed_44/run_manifest.json`
- `outputs/damage_dataset/benchmark_summary.json`
- `outputs/crisisMMD/benchmark_summary.json`